In [1]:
```
import cv2
import numpy as np
from skimage.morphology import skeletonize
from skimage import data
from skimage.util import invert
import skimage
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import plotly
import plotly.express as px
import plotly.graph_objects as go
```

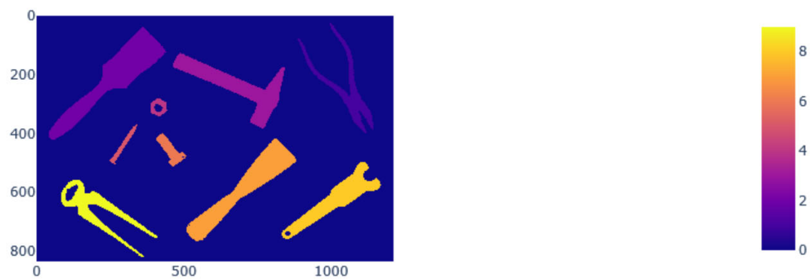
Einlesen des Bildes mit den Werkzeugen aus den BV-Lehrbuch

In [2]:
```
img = 255 - cv2.imread(r'C:\workspace\ComputerVision\Source\017_Kapitel_RegionenInB
```

Markieren der Regionen im Binärbild

In [3]:
```
labels = skimage.measure.label(img).astype('uint8')
```

In [4]:
```
fig = px.imshow(labels)
fig.show()
```



Feature Vektor für jedes Objekt erzeugen und in eine große Matrix speichern. Die Spalten sind die Features, die Zeilen die Objekte. nummerierung der Objekte gemäß der labels

In [5]:
```
def calcHuMoments(labels):
    numberOfObjects = np.max(labels)

    huMomentsList = []

    for region in np.arange(1, numberOfObjects+1):
        # isolate image
        img_tmp = np.zeros(labels.shape, dtype='uint8')

        img_tmp[labels == region] = 1

        # calculate hu Moment
        moment_central = skimage.measure.moments_central(img_tmp)
        moment_normal = skimage.measure.moments_normalized(moment_central)
```

```
        moment_Hu = skimage.measure.moments_hu(moment_normal)
        moment_Hu_log =- np.sign(moment_Hu)*np.log(np.abs(moment_Hu))
        huMomentsList.append(list(moment_Hu_log))
        pass

    huMoments = np.array(huMomentsList)
    return huMoments
```

In [6]:
```
huMoments = calcHuMoments(labels)
```

Objekt laden und Hu Momente verechnen

In [7]:
```
# analyze Object
obj = cv2.imread(r'.\images\Bzange_binary.png', cv2.IMREAD_ANYDEPTH)
obj = obj/255
```

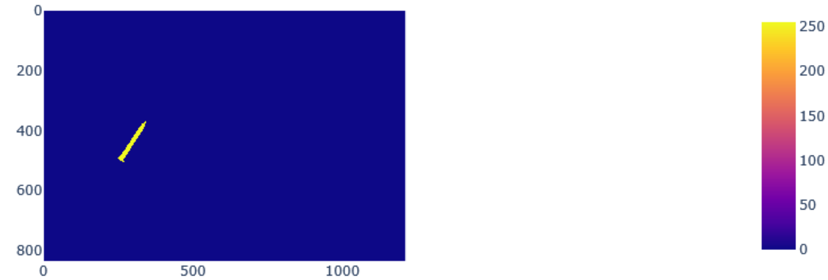
In [8]:
```
objMoments = calcHuMoments(obj)
diff = huMoments - objMoments
distance = np.linalg.norm(diff, axis =1)

objectNumber = np.argmin(distance)+1
```

In [9]:
```
## Bild freistellen
obj_detected = np.zeros(labels.shape)
obj_detected[labels == objectNumber] = 255
```

In [10]:
```
fig = px.imshow(obj)
fig.show()

fig = px.imshow(obj_detected)
fig.show()
```


0
200
400
600
800
0
500
1000
250
200
150
100
50
0